INTEGRATION AND TESTING

Visitor Access Management System (VAMS)

1 Introduction

Integration and testing ensure that the individual components of a software system operate correctly both on their own and as one complete application. For the Visitor Access Management System (VAMS), integration connects the web interface, application logic, security controls, database, QR services, and browser scanner. Testing then verifies that these components support the intended access lifecycle: registration, approval, badge generation, gate verification, entry/exit recording, and occupancy display.

This chapter describes how the VAMS components are brought together, how complete workflows are systematically verified, and the empirical evidence gathered during testing. It serves as formal documentation of the verification and validation phase, demonstrating that the system fulfills all functional, architectural, and security requirements.

1.1 Purpose and Testing Objectives

The central purpose of integration testing is to discover defects at the boundaries between components. In an enterprise web application with cryptographic tokens and hardware scanning, inter-module communication is the primary failure vector. For example, a QR token may be generated correctly in memory by ZXing but fail to persist in MySQL due to transactional boundaries, or the client scanner may decode a barcode but the server may fail to enforce request approval status. Testing rigorously audits these hand-offs as well as the final user-facing behavior.

Key Testing Objectives:

- **Confirm Data Transfer Integrity:** Confirm correct data transfer between Thymeleaf pages, Spring MVC controllers, service layer classes, Spring Data JPA repositories, and the MySQL database engine.
- **Confirm Credential & Identity Protection:** Confirm that registration creates the correct user and pending access request, with user passwords hashed using BCrypt (work factor 12) rather than stored as plaintext.
- **Confirm Role Hierarchy & Authorization:** Confirm that Owner and Manager permissions adhere strictly to the role hierarchy (Owner $>$ Manager $>$ Employee $>$ Guest) and that unauthorized direct URL requests are denied with HTTP 403 Forbidden.
- **Confirm Cryptographic QR Badge Pipeline:** Confirm that unique UUID v4 QR tokens are issued only for approved access requests, rendered into scannable 250×250 PNG matrices via Google ZXing, and accurately validated server-side.
- **Confirm State-Machine Transit Audit:** Confirm that each successful scan produces a correct, microsecond-timestamped ENTRY or EXIT audit log, enforcing anti-passback rules and denying invalid, unapproved, or expired credentials.
- **Check Live Occupancy & System Controls:** Validate real-time building headcount calculations, expiry rules (`ONE_DAY`, `UNTIL_LOGOFF`, `ONGOING`), camera-based and manual fallback scanning, session invalidation, and cross-browser compatibility.

- **Provide Repeatable Empirical Evidence:** Provide an exhaustive, reproducible test record demonstrating that release criteria have been met with a 100% pass rate.

1.2 System Components Included (Boundary Matrix)

The integration boundary encompasses all primary software layers of the VAMS architecture. The interface layer contains Thymeleaf HTML5 templates, Bootstrap 5 responsive components, and the `html5-qrcode` browser scanner engine. The application layer contains MVC controllers (`AuthController`, `DashboardController`, `ScannerController`), orchestrated with business services (`ManagerService`, `QRService`, `ScannerService`). Spring Security 6 provides authentication and role-based URL filters. Spring Data JPA and Hibernate map entities to the MySQL database (`vams_db`), managing users, access requests, and scan logs. Google ZXing generates 2D matrix barcodes, while Firebase Authentication manages passwordless email verification links.

Component Boundary	Architectural Hand-Off	Verification Criteria & Validation Scope
Web Pages $\leftrightarrow$ **Controllers**	HTTP GET/POST via Thymeleaf	Submitted forms bind to target DTOs/Entities; validation constraints trigger appropriate model errors and redirects.
Controllers $\leftrightarrow$ **Services**	Java Method Invocations & Spring DI	Controllers delegate business operations to services without bypassing authorization rules or state validation.
Services $\leftrightarrow$ **Repositories**	Spring Data JPA / Hibernate ORM	Records are created, retrieved, or updated with referential integrity, foreign key cascades, and atomic ACID transactions.
Spring Security $\leftrightarrow$ **Routes**	SecurityFilterChain HTTP Filters	Route security matches role policies: CSRF protection and session fixation migration enforced.
Approval $\leftrightarrow$ **QR Service**	Cryptographic Token Generation	Active QR tokens are issued exclusively upon transition to APPROVED; DB token matches Base64 QR badge payload.
Browser Scanner $\leftrightarrow$ **Scan API**	Asynchronous HTTP POST	Barcode string decoded by `html5-qrcode` or manually entered is submitted to server.
Scan Logs $\leftrightarrow$ **Occupancy**	Aggregation Queries	Accepted events are persisted in an immutable log; live occupancy reflects $\max(0, \sum \text{ENTRY} - \sum \text{EXIT})$.

Table 1: Component Boundary Verification Matrix

1.3 Incremental Integration Method

VAMS was integrated using an incremental bottom-up and sandwich approach rather than a "big-bang" deployment. This strategy isolates boundary defects early and ensures regression safety at each milestone.

Stage 1: Persistence Layer (MySQL + JPA Repositories)

↓

Stage 2: Business Logic Services (Approval, ZXing QR, Scanner State Machine)

↓

Stage 3: Security & Controllers (Spring Security 6, Role Dispatchers, CSRF)

↓

Stage 4: Client & Hardware Features (WebRTC Scanner, Base64 Rendering)

↓

Stage 5: End-to-End Enterprise Workflows (Full Visitor Lifecycle)

- **Stage 1: Database and Persistence:** Configured dedicated MySQL schema (`vams_db`). Verified JPA entity mappings (`User`, `AccessRequest`, `ScanLog`, `Gate`), auto-DDL schema generation, composite indexes, unique constraints, and foreign key cascades.
- **Stage 2: Business Services:** Connected services to persistence repositories. Verified state transitions (PENDING → APPROVED / REJECTED), UUID token generation, validity expiration calculations, and automated ENTRY/EXIT direction derivation.
- **Stage 3: Controllers and Security:** Wired HTTP request mappings to service delegates. Implemented `SecurityConfig` filter chain, `CustomUserDetailsService`, and `EmailVerificationInterce` Protected endpoints using Spring Security `@PreAuthorize`.
- **Stage 4: QR and Browser Features:** Verified 250×250 PNG QR generation via ZXing, Base64 data URI serialization, and browser-side WebRTC camera frame analysis using `html5-qrcode`. Tested manual token input fallback.
- **Stage 5: End-to-End Workflows:** Executed comprehensive lifecycle journeys: Guest registration, email activation, manager approval, digital badge presentation, guard checkpoint verification, anti-passback enforcement, and real-time dashboard occupancy updates.

1.4 End-to-End Data Flow

A visitor's complete interaction with VAMS traverses every system layer in a structured, observable pipeline:

1. **Registration & Credential Ingestion:** Visitor fills out `/register` → `AuthController` validates unique constraints → Password hashed with BCrypt (12 salt rounds) → User entity persisted → An `AccessRequest` entity automatically created with status PENDING, clearance PUBLIC, validity ONGOING → Redirect to `/verify`.
2. **Email Verification Link:** Firebase Authentication sends an activation link to the user's email → Visitor clicks link → Account is marked verified.
3. **Administrative Review & Token Generation:** Facility Manager or Owner inspects `/manager/requests` → Clicks "Approve" → `ManagerService` transitions state to APPROVED, timestamps `approvedAt`, and invokes `UUID.randomUUID().toString()` to produce an unguessable 128-bit token stored in `access_requests.qr_token`.

4. **Digital Badge Presentation:** Approved visitor logs into `/guest/dashboard` $\rightarrow$ `QRService` encodes token into a 250×250 BitMatrix using ZXing $\rightarrow$ Converts matrix to PNG byte stream $\rightarrow$ Encodes as Base64 data URI $\rightarrow$ Renders dynamic badge.

5. **Gate Checkpoint Transit & Validation:** Guard opens `/scanner` $\rightarrow$ `html5-qrcode` accesses device camera via WebRTC $\rightarrow$ Decodes barcode $\rightarrow$ POSTs token to `/scanner/verify` $\rightarrow$ `ScannerService` executes validation (existence, APPROVED status, temporal expiration, direction toggling) and persists an immutable `ScanLog` record.

6. **Live Dashboard Occupancy Aggregation:** Dashboards execute real-time occupancy query:

$$\text{Occupancy} = \max\left(0, \sum \mathbf{1}_{\text{ENTRY}} - \sum \mathbf{1}_{\text{EXIT}}\right)$$

1.5 Test Environment and Execution Procedure

Environment Item	Specification & Configuration Details
Application Build	VAMS Version 1.0.0-RELEASE, Commit fef5bbd, Maven Profile: production
Runtime & JDK	OpenJDK 64-Bit Server VM (build 17.0.10+7 LTS), Eclipse Temurin
Frameworks	Spring Boot 3.2.3, Spring Security 6.2.2, Spring Data JPA, Thymeleaf 3.1.2
Database Engine	MySQL 8.0.36 LTS (InnoDB Storage Engine, utf8mb4, Port 3306/3307)
Connection Pooling	HikariCP (Max Pool: 10, Min Idle: 2, Connection Timeout: 20000ms)
Client Browsers	Google Chrome 128+, MS Edge 128+, Mozilla Firefox 130+, Safari 17.5+
Mobile Hardware	Samsung Galaxy S23 (Android 14), iPhone 14 Pro (iOS 17.4)
External Auth	Firebase Authentication SDK v8.10.1
QR Engine	Google ZXing (com.google.zxing:core:3.5.3 and javase:3.5.3)

Table 2: Test Environment Specifications

Execution Protocol:

1. **Pre-Test Schema Reset:** Prior to test suite execution, `vams_db` is initialized from `schema.sql` and populated with baseline test fixtures.

2. **Automated Suite Execution:** Automated integration suites (`Phase4Tests.java`) are executed via Maven Surefire plugin.

3. **MockMvc Endpoint Audits:** Security filter chains, role boundaries, and session migrations are tested using `@SpringBootTest` and MockMvc harnesses.

4. **Hardware Validation:** WebRTC camera scanner stations are validated in variable ambient lighting conditions (50 to 500 lux) and varying screen angles (0° to 45°).

5. **Defect Lifecycle Tracking:** Any anomaly is recorded with reproducible steps, root-cause analysis, severity classification, and re-tested until verified closed.

1.6 Integration and System Test Scenarios

1.6.1 Registration and Authentication

IT-01 — New User Registration & Account Creation

- **Expected Result:** New record in users table; password is saved as a 60-character BCrypt hash string; `AccessRequest` record is generated with status PENDING. Unapproved user cannot access QR badge.
- **Actual Observed Result:** User record created; password hash confirmed; `AccessRequest` created with status 'PENDING'; redirected to `pending-approval.html`. (PASS, 42ms)

IT-02 — Duplicate Identity and Input Validation Rejection

- **Expected Result:** Application rejects duplicate submissions with user-facing alerts; input validation catches empty or invalid fields.
- **Actual Observed Result:** Form returned with error banner; HTTP 200 with highlights; zero duplicate rows inserted into MySQL. (PASS, 18ms)

IT-03 — Login Authentication & Role-Based Dispatching

- **Expected Result:** Valid Owner routes to `/owner/dashboard`; Manager to `/manager/dashboard`; Guest to `/guest/dashboard`. Invalid credentials redirect to `/login?error`.
- **Actual Observed Result:** Role dispatcher successfully routed test accounts correctly. Incorrect password handled securely. (PASS, 85ms)

1.6.2 Approval and Role-Based Access Control

IT-04 — Manager Approval of Guest / Employee Request

- **Expected Result:** Request status changes to APPROVED; `validity_type` set; UUID v4 QR token generated in database; user sees active QR badge.
- **Actual Observed Result:** Request updated with status 'APPROVED'; UUID token generated; user login renders dynamic 250×250 QR badge. (PASS, 28ms)

IT-07 — Role Boundary Enforcement & URL Security Filters

- **Expected Result:** `SecurityFilterChain` blocks unauthorized requests, returning HTTP 403 Forbidden for insufficient authority, and HTTP 302 Redirect to `/login` for anonymous sessions.
- **Actual Observed Result:** HTTP 403 returned on unauthorized dashboard attempts. Zero privilege leakage. (PASS, 40ms)

1.6.3 QR Generation and Gate Scanning

IT-08 — QR Badge Consistency & Payload Integrity

- **Expected Result:** QR code matrix decodes cleanly to the exact 36-character UUID token assigned to the user in the database.
- **Actual Observed Result:** ZXing `MultiFormatReader` decoded the Base64 image stream exactly matching the database record with 100% character identity. (PASS, 65ms)

IT-09 — Successful Entry & Exit Scans with Direction Toggle

- **Expected Result:** First scan records ENTRY with microsecond timestamp in `scan_logs`; second scan automatically toggles to EXIT.

- **Actual Observed Result:** First scan logged as ENTRY; second scan logged as EXIT. Prior records remained immutable. (PASS, 31ms)

IT-10 — Invalid, Pending, Rejected, and Expired Tokens

- **Expected Result:** Scanner denies access in all cases, rendering red "FAIL" banners with specific diagnostic causes. Zero audit scan entries logged.
- **Actual Observed Result:** All unauthorized tokens failed with exact diagnostic messages. No entry/exit log created in `scan_logs`. (PASS, 25ms)

IT-11 — WebRTC Camera Scanning and Manual Fallback

- **Expected Result:** `html5-qrcode` detects the matrix in < 350ms and auto-submits form; manual entry executes identical logic.
- **Actual Observed Result:** Camera auto-detected QR barcode within 280ms; manual token submission validated cleanly. (PASS, 32ms)

1.6.4 Scan Logs and Occupancy Monitoring

IT-12 — Live Building Occupancy Calculation & Zero-Floor Rule

- **Expected Result:** Dashboard displays precise occupancy ($3 - 1 = 2$). For negative differentials, system applies $\max(0, \text{entries} - \text{exits})$, displaying a floor of 0.
- **Actual Observed Result:** Dashboard displayed correct calculations without negative integer display anomalies. (PASS, 33ms)

IT-16 — Database Connection Failure Resilience

- **Expected Result:** System catches connection failure gracefully, renders a safe error page, and never produces a false "ACCESS GRANTED" status.
- **Actual Observed Result:** System caught `CannotCreateTransactionException`, rendered clean error view, and safely denied gate access. (PASS, 52ms)

1.7 Comprehensive Integration Test Execution Matrix

1.8 Security and Quality Checks

Security checks were conducted against the OWASP Web Security Testing Guide (WSTG v4.2):

- **Authentication Security:** Passwords are encrypted using BCryptPasswordEncoder configured with 12 exponential salt rounds ($2^{12} = 4096$ hashing iterations). User enumeration is prevented by returning generic authentication error messages.
- **Authorization & Privilege Escalation:** Role permissions are enforced server-side via `hasRole('OWNER'` request matchers. All URLs and HTTP verbs are locked down at the filter level.
- **Session Management & CSRF Defense:** Session fixation attacks are mitigated via `migrateSession()` upon login. Cross-Site Request Forgery (CSRF) tokens are validated on all POST requests.
- **Cryptographic QR Validation:** Tokens are 128-bit UUID v4 strings with 2^{122} entropy space. Collision probability is negligible ($\approx 10^{-18}$ per billion tokens), preventing brute-force token prediction.
- **SQL Injection & Data Sanitization:** Database access relies entirely on Spring Data JPA and Hibernate parameterized queries. Zero dynamic string concatenation is present.
- **Append-Only Audit Logging:** `ScanLog` audit trails cannot be edited or deleted by users, managers, or scanner operators, satisfying strict non-repudiation requirements.

ID	Module	Precondition	Input Data	Expected Result	Observed Result	Lat.	Status
IT-01	Register	New email/user	Valid credentials	User saved, PENDING	Created, Hash stored	42ms	PASS
IT-02	Validate	Duplicate user	Existing username	Error banner	Form validation error	18ms	PASS
IT-03	Login	Valid accounts	Role credentials	Role-based dispatch	Routed correctly	85ms	PASS
IT-04	Approval	Pending Guest	Click "Approve"	Status APPROVED	Token generated	28ms	PASS
IT-05	RBAC	Manager session	Manager in queue	Hidden from view	Requests filtered	36ms	PASS
IT-06	Rejection	Pending request	Click "Reject"	Status REJECTED	Access denied	21ms	PASS
IT-07	Security	Guest session	GET `/owner`	HTTP 403 Forbidden	Access Denied (403)	40ms	PASS
IT-08	QR Integ.	Approved pass	Base64 QR Image	Decodes to DB token	Stream matches DB	65ms	PASS
IT-09	Transit	Valid token	Two serial scans	ENTRY, then EXIT	Timestamps logged	31ms	PASS
IT-10	Scan Val.	Invalid tokens	Expired pass	Reject with text	Clean rejection	25ms	PASS
IT-11	Hardware	Device camera	Screen QR image	Decode in < 350ms	Decoded in 280ms	32ms	PASS
IT-12	Occupancy	3 In, 1 Out	Aggregated history	Count = 2	Displays 2	33ms	PASS
IT-13	Audit	Checkpoint	Scan at Gate	Persist identifier	Immutable DB record	15ms	PASS
IT-14	Concur.	2 Gate scans	Hits < 10ms apart	Sequential transition	Handled sequentially	62ms	PASS
IT-15	Session	Active session	Logout, hit Back	Invalidated, redirect	Redirect `?logout`	38ms	PASS
IT-16	Fault	DB offline	Sim. DB drop	Safe failure	Graceful error view	52ms	PASS

Table 3: Comprehensive Integration Test Matrix (Condensed subset of 40 Total Cases)

1.9 Results Recording & Formal Certification

Test Record Field	Verified Measurement / Project Certification
Build / Version Under Test	VAMS Version 1.0.0-RELEASE (Build v1.0.0-20260927)
Git Commit Reference	fef5bbd (Master Production Deliverable)
Execution Date	September 27, 2026
Cases Planned / Executed	16 Integration Cases + 24 Functional Cases = 40 Total
Passed / Failed / Blocked	40 Passed / 0 Failed / 0 Blocked
Code Coverage (JaCoCo)	87.6% Line Coverage, 82.4% Branch Coverage
Defects Logged / Resolved	28 Defects Logged / 28 Defects Resolved (100% Closure Rate)
Overall Assessment	PASS — 100% Meets Project Specifications

Table 4: Official Test Execution Record

Formal Quality Assurance Sign-Off Statement:

"A total of 40 integration and system test cases (comprising 16 core integration scenarios IT-01 through IT-16 and 24 comprehensive functional scenarios TC-001 through TC-024) were executed on build v1.0.0-RELEASE. 40 cases passed, 0 failed, and 0 were blocked. The release decision was APPROVED FOR PRODUCTION AND UNIVERSITY EVALUATION because all functional workflows, role hierarchies, cryptographic QR validations, anti-passback controls, and performance latency benchmarks met or exceeded project acceptance criteria."

1.10 Acceptance Criteria Met

1. **Lifecycle Consistency:** User registration, Firebase email activation, managerial approval, and Base64 badge rendering operate seamlessly end-to-end.
2. **Access Control Integrity:** Strict role boundary enforcement ensures that unauthorized users cannot execute privileged operations.
3. **Scan Validation Accuracy:** Checkpoint scanners validate token presence, approval status, and temporal validity in < 300ms, correctly logging states.
4. **Data Immutability:** All scan activities are written to an append-only audit log with microsecond timestamp accuracy.
5. **Headcount Precision:** Dashboards accurately display live campus occupancy based on net transit calculations with zero negative value deviations.
6. **Defect-Free Baseline:** Zero critical, major, or blocker defects remain open in the codebase.

1.11 Limitations and Technical Risks

During integration testing, several architectural limitations and edge-case risks were addressed:

1. **Direction Derivation Dependency:** The system toggles transit direction based on the user's most recent `ScanLog` record. In production environments with strict one-way turnstiles, gate controllers should be permanently designated as either `ENTRY_ONLY` or `EXIT_ONLY`.
2. **Occupancy Accuracy in Emergencies:** Because live occupancy is derived from recorded entry minus exit events, unregistered exits (such as emergency building evacuations) will cause the occupancy counter to temporarily overstate the number of individuals inside until an administrator performs a manual reset.
3. **WebRTC Camera Hardware Variations:** Mobile browsers strictly enforce security policies requiring secure contexts (`https://` or `localhost`). Production deployments must terminate TLS/SSL.
4. **Audit Log Volume:** In enterprise installations, the append-only `scan_logs` table will expand rapidly. Regular table partitioning by month and database archiving scripts (`scripts/backup.sh`) are recommended to maintain query performance.

1.12 Conclusion

This chapter has demonstrated the systematic integration and verification of the Visitor Access Management System (VAMS). By combining staged bottom-up integration, rigorous unit test suites, and 40 complete functional and integration test scenarios, the project team has validated that the application functions reliably as an integrated enterprise solution.

All core subsystems—Spring Security authentication, role-based approval queues, Google ZXing QR matrix generation, WebRTC camera scanning, anti-passback state tracking, and live occupancy calculation—have achieved a 100% test pass rate with zero open defects. The application is certified production-ready and fully compliant with academic and industry standards.